

SRW3 Phase 1A — LLVM Retarget and Proof-Boundary Experiment

Research report — backend portability, artifact-integrity repair, and the three prover boundaries

Phase	1A (LLVM retarget)
Starting state	Phase 0-D, Theorem-Closure Pass (closed)
Mandate	Retarget KEVM/SRW3 to the LLVM backend; determine whether LLVM removes any Haskell-backend proof/packaging boundary; preserve semantic equivalence; repair artifact provenance
Starting commit	80448f22cd9f24e6598565992db1e447c71cd4db (branch phase1-start)
Toolchain	K v7.1.337, KEVM v1.0.921, blockchain-k-plugin 207ae5121e5178a09742ed746f2d15e34b1750cc (K sources), Z3 4.13.3, clang/lld 15.0.6-4+b1 (new, §3)
Date	2026-09-29

Headline result. The LLVM backend compiles and executes the complete SRW3 stack — the abstract semantics (13/13 demos semantically equivalent to Haskell, cell-by-cell) and the full KEVM binding (3/3 demos structurally identical, byte-equal normalized decision cells) — while **removing none of the three proof boundaries**. Proof closure remains Haskell-backend-only: kprove rejects the LLVM backend at its argument gate (Backend llvm does not support execution. Supported backends are: [haskell]). Tier-3 universal reconciliation remains **STATEMENT FORMALIZED; NOT YET MECHANIZED**. The Phase-0 evidence classifications are unchanged; the regression matrix v4 is zero-regression. The Phase-0 concern that the KEVM LLVM link requires building libkrypto.a did **not** materialize for the K-source-only plugin path: the binding kompiles under LLVM in 85 s / 2.28 GiB and executes correctly.

1. Phase-0 starting state

Phase 1A takes the Phase-0 theorem-closure final state as its fixed starting point. That state existed at the end of Phase 0 only as an uncommitted working tree, so the first action of this phase was the canonical provenance repair of Part I.B (below). The repaired chain is:

```
final source tree (srw3-kevm/)
-> git commit 80448f2 (branch phase1-start, parent 69859b1)
-> reproducible git bundle srw3-work.bundle (sha256 92481f29d1fcb0cad0e6d73db9899c0c9eaf9d6f5b06676078b67ee6eb0d7021)
-> commit hash recorded in this report (§1) and in transcripts/provenance/PROVENANCE.md
```

The full commit lineage is a857618 (main; byte-identical baseline

SRW3_Phase_0D_Implementation_v1.0.zip, sha256

2db8669f8aedd5bceff762bfd4a4901ddc9cb3f3e6387364e0abe2ac8e62955) → 69859b1 (kevm-changes; recorded K v7.1.337 compatibility fixes, pristine untouched) → 80448f2 (phase1-start; the complete research workspace: k/srw3.k, the ghost artifact proofs/induction_ghost.k with its frozen fidelity manifest, all proof sources, the KEVM binding, all transcripts, and the Phase-0 report).

Kompile-output directories are excluded from the commit as reproducible derivatives (srw3-kevm/.gitignore documents the exclusion list); git status at the commit is clean, and the full delta against kevm-changes (60 files) is archived in transcripts/provenance/.

The scientific starting state is exactly the Phase-0 closure: Claims 1–3 and 4a/4b PROVED BY K; the full $\forall n$ lineage-length safety theorem MECHANIZED over the ghost-instrumented semantics (GH-T2/GH-M, #Top); tier-1/tier-2 reconciliation PROVED BY K; tier-3 STATEMENT FORMALIZED; NOT

YET MECHANIZED; cryptographic lineage NOT YET MECHANIZED; the KEVM binding DEMONSTRATED BY KEVM; network-level commitment enforcement REQUIRES CLIENT/PROTOCOL SUPPORT.

2. Artifact-integrity repairs

2.1 Ghost source-fidelity checker (Part I.A)

Deficiency. The Phase-0 faithfulness check (`audit_ghost_faithful.py`) proved only that the 507 baseline lines occur in order inside the artifact after deleting every line containing the `//GH0ST:` marker. This is a one-sided subsequence property: it verifies $\text{baseline} \subseteq \text{kept}$ but never verifies that `kept` contains nothing beyond the baseline plus declared additions. In particular it silently tolerated (i) six unmarked comment additions inside the copied body, and (ii) the entire 290-line appended extension region (SRW3-GH0ST-OBS, SRW3-GH0ST-CLAIMS — 199 lines of live ghost code), which are real semantic additions but were outside any verified equation. It also did not distinguish header/comment occurrences of the marker from executable additions.

Repair. The replacement checker (`scripts/audit_ghost_exact.py`, frozen manifest `proofs/ghost_manifest.json`) verifies the explicit equation:

```
baseline_semantics + exactly_declared_ghost_additions = ghost_instrumented_semantics
```

The manifest freezes: the baseline slice (`k/srw3.k` lines 24–530, 507 lines, sha256 4586c1cb...); the declared header block; **seven declared insertion blocks** (anchor line, occurrence, exact text, per-line CODE/COMMENT classification, marker flag — 13 marked live-code lines + 6 declared comment lines, including the blank separator line); and the declared appended extension region (290 lines, frozen by sha256 and line count). Verification then checks (0) the baseline slice is unchanged since freeze, (1) whole-artifact byte-identity against the frozen artifact hash, (2) header byte-identity, (3) extension-region byte-identity, (4) a deterministic reconstruction — walking the baseline and inserting each declared block at exactly its declared occurrence of the anchor — equals the copied body byte-for-byte (526 reconstructed lines), and (5) classification accounting. Any missing, reordered, or altered baseline line, any undeclared addition inside the copied body, or any drift in the extension region is pinpointed with a divergence class (`missing`, `missing-or-reordered`, `undeclared-addition`, `altered`).

Accounting (the mandated distinction). Total `//GH0ST:` occurrences: **14** — of which **1** is the explanatory mention inside the artifact header comment (NOT executable), and **13** are marked live-code additions in the copied body. Declared comment additions inside the body: **6** (5 explanatory + 1 blank separator), all unmarked and declared. Extension-region code lines: **199** (the ghost function definitions and claims modules — the extension region proper). Undeclared executable additions inside the copied body: **0**. Verdict: **PASS — exact byte-faithfulness established**, archived in `transcripts/audit/ghost_fidelity_exact.txt`.

Detection self-test. The checker ships with a `selftest` mode that mutates a copy of the artifact four ways — missing baseline line, altered baseline line, reordered baseline lines, undeclared addition inside the copied body — and requires the structural reconstruction check itself (not the trivial whole-file hash) to fail on each. All four mutations are detected; the test transcript is part of the audit record. The semantics files were not modified in any way: the checker was improved, the artifact was not.

2.2 Final Git provenance (Part I.B)

Deficiency. The Phase-0 artifact bundle's git history ended at 69859b1 (compat fixes); the theorem-closure final state — including the ghost artifact whose `#Top` verdicts the Phase-0 report cites — was never committed. The bundle therefore could not reconstruct the tree that the report described.

Repair. A new branch `phase1-start` was created from `kevm-changes` and the complete final source tree was committed as `80448f2` with a canonical message. The reproducible bundle `srw3-work.bundle` now carries all three refs (`main`, `kevm-changes`, `phase1-start`; `git bundle verify` reports a complete history; bundle sha256 `92481f29...`). Reconstruction is `git clone srw3-work.bundle && git checkout phase1-start`. Recorded per the mandate: final commit hash, parent commit, branch, clean git status, the full git diff against `kevm-changes` (`stat + complete patch` in `transcripts/provenance/`), the bundle sha256, the pinned K/KEVM/plugin revisions, and the baseline sha256. No historical claim was rewritten — `main` and `kevm-changes` are untouched; the final state is a new descendant commit.

3. LLVM environment

The pinned Phase-0 toolchain was restored after a container restart (workspace loss of `tools/`, `srw3-work/`, and `srw3-baseline/`; all three recovered from the snapshot at `/tmp/my-project/tools` and the verified Phase-0 artifact bundle, with `srw3-work` reconstructed by cloning its own git bundle and `srw3-baseline` from the bundle's byte-faithful copy). All versions are unchanged: K v7.1.337 (llvm-backend 0.1.140), KEVM v1.0.921 source, blockchain-k-plugin K sources at the pinned SHA (restored from GitHub into the `kproj` include path — `plugin/krypto.md` is required by the `serialization.md` chain), Z3 4.13.3.

Minimum requirements determined for an LLVM-backend definition. Beyond the Haskell path, `kompile --backend llvm` needs exactly: (1) K's `llvm-kompile` toolchain (present in the K .deb, incl. `kllvm` static libraries); (2) a C++17 compiler — K 7.1.337 hardcodes `/usr/bin/clang++-15`; (3) the `lld` linker (K passes `-fuse-ld=lld`); (4) dev-symbol availability for four host runtime libraries the interpreter links (`libtinfo`, `libmpfr`, `libjemalloc`, `libunwind`); (5) the `mpfr.h` header when `--enable-search` is used (the `kllvm` runtime includes it). No large optional component was built; the C `libkrypto` remains unbuilt (as in Phase 0 — no keccak-dependent opcode is exercised).

Documented deviations (toolchain only; no semantics touched).

deviation	detail
clang-15 15.0.6-4+b1	Debian pool, exact source-version match with the already-pinned <code>libllvm15</code> ; extracted to <code>tools/clangpkg</code> without root
K wrapper override	<code>llvm-kompile-clang</code> patched to honor <code>LLVM_KOMPILE_CXX</code> (one line; original preserved as <code>llvm-kompile-clang.orig</code>); Phase-1A addition recorded in <code>tools/env.sh</code>
lld-15 + libclang-cpp15	same pool version, extracted to <code>tools/clangpkg</code>
lib dev symlinks	<code>tools/hostlibs/lib/lib{tinfo,mpfr,jemalloc,unwind}.so</code> → host runtime <code>.so.N</code> (no root; <code>LIBRARY_PATH</code>)
mpfr.h	<code>libmpfr-dev</code> extracted to <code>tools/hostlibs/usr/include</code> (CPATH) — needed only for <code>--enable-search</code> builds

4. Abstract LLVM results

`kompile srw3.k --backend llvm --main-module SRW3-GATE --syntax-module SRW3-GATE` succeeds on the **unmodified** Phase-0 semantics file: exit 0, 12.6 s, 377.1 MiB peak. The invocation differs from the Haskell one only in the backend flag and in explicitly passing the main/syntax module names (the same module names the Haskell definition records). No hidden source rewrite was required; the warnings are recorded (`transcripts/llvm_kompile_sr3.log`) and are benign — non-exhaustive-match and unused-variable warnings of the same classes the definition produces on the Haskell path (the semantics uses `[owise]` completions by design). A search-enabled LLVM build (`--enable-search`) also succeeds (15.4 s, 458.8 MiB).

Concrete execution — 13/13 abstract demos. The LLVM interpreter runs the full demo suite. Comparison against a fresh Haskell run of the same 13 programs is **semantic, not exit-code based**: for every demo the comparator extracts the decision-relevant cells — `result`, `committed`, `head`, `lineageNext`, `lineage`, `prospective` — from the final `<srw3>` block and requires equality. All 13 demos match on every cell (`transcripts/audit/llvm_demo_equivalence.txt`). The suite covers the decision taxonomy end-to-end: minimal reject, negative-model accepts (CM1), two-step accept, hidden-write commits under trust (CM7), co-edge defense-in-depth reject, effect-incompleteness block (CM4), unauthorized reject-auth, honest chain commit, stale-price reject, declared-read stale reject, three-way lineage rejects, and the pairwise-only hole (CM6).

Reasoning capability probes. LLVM `krun --search` executes concrete branch enumeration only (deterministic demo → one concrete solution). This is the extent of LLVM "reasoning": there is no symbolic evaluator in the LLVM backend, which is the structural fact behind every result in §5.

5. Three prover-boundary experiments

Each boundary was first reproduced as a minimal, standalone artifact, then run under both backends. The LLVM column is uniform and definitive: `kprove` cannot even be pointed at the LLVM backend — every attempt terminates at the same argument gate, before any spec processing:

```
[Error] Critical: Backend llvm does not support execution. Supported backends are: [haskell]
[Error] Critical: Invalid backend: llvm. It should be one of [kore, haskell]
```

(`transcripts/audit/llvm_kprove_probe1.txt`, and per-boundary files `b1_llvm.txt`, `b2_llvm.txt`, `b3_llvm.txt`.)

5.1 Boundary 1 — symbolic Map evaluation

Reproduction (`proofs/boundaries/b1_symbolic_map.k`): a total recursive fold sum over `Map` (`.Map` base case; `K |-> V M` destructuring), with two claims: `B1-SYM` (the `Map` pinned by an `M ==K (1 |-> 10 2 |-> 20)` premise) and `B1-FREE` (a fully symbolic constructor-free `Map`).

Haskell result. `B1-SYM` closes — but only trivially (`WarnTrivialClaim`: "proven without rewriting"): the pin is discharged by the implication check, and the recursive fold is never exercised. `B1-FREE` is **STUCK** (`WarnStuckClaimState`) with the residual sum `(... m: M ) ==Int S` — the canonical unreduced recursive application over a symbolic constructor-free `Map`. This is exactly the Phase-0 blocker class ("recursive map lookups over symbolic constructor-free Maps do not reduce sufficiently for the proof packaging"); note the ghost theorem worked with this boundary by using shared-variable residual forms (`GH-SOUND-I`) rather than against it.

LLVM result. Not applicable — attempt rejected at the backend gate (§5 preamble). The theorem was not redesigned first, per the mandate.

5.2 Boundary 2 — destination-variable linkage

Reproduction (`proofs/boundaries/b2_destination.k`): a one-step definition updating `<st>` and `<log>`, with `B2-OMIT` (claim destination mentions only `<k>`) and `B2-EXPL` (explicit `?-existential` destination cells over all changing cells).

Haskell result. `B2-OMIT` is **STUCK** with exactly the Phase-0 residual class: the configuration unifies with the destination but the implication check fails on the generated dangling variables — verbatim `_Gen3 #Equals _Gen3 +Int 1` (the `?YB` class of Phase 0, with generated names). `B2-EXPL` **closes**. This confirms the boundary is a property of Kore destination handling, reproducible in a 34-line file.

LLVM result. Not applicable — backend gate (§5 preamble).

5.3 Boundary 3 — symbolic-set induction (tier-3 universal)

Reproduction. The exact unweakened artifact proofs/tier3_universal.k — $\text{wfCoverage}(H, FD, S) \rightarrow \text{gateOk_faithful}(H, S) = \text{implementedGate}(H, FD, S)$ with universal $\forall H \forall FD \forall S$ (T3-U), the incremental symbolic-FD form (T3-FD), and the authorization lemma (T3-A). Nothing was weakened, reduced, or re-quantified.

Haskell result. Fresh re-run: all three claims STUCK (one `WarnStuckClaimState` each, residuals archived in `transcripts/audit/tier3_universal.txt`). The blocker is unchanged: recursive functions over symbolic Sets do not reduce and structural induction over symbolic aggregates is unavailable in the existing proof setup.

LLVM result. Not applicable — backend gate (§5 preamble).

5.4 Scientific interpretation of Part V

The three boundaries are **properties of the Haskell backend's symbolic machinery** (kore-repl/z3 proof packaging), not of the K language or of the SRW3 semantics. The LLVM backend exposes no symbolic engine at all, so "retargeting" a proof to LLVM is not merely difficult — it is category-error: there is nothing for the proof to run on. Changing backend therefore **cannot** move any of the three boundaries, and empirically does not: each boundary was reproduced fresh under Haskell and recorded, and the LLVM column is uniformly "no symbolic backend". The scientific value of the experiment is precisely this negative space: it establishes that the tier-3 gap and the $\forall n$ packaging constraints are tool-logic-frontend facts that a backend swap cannot address, and that any future attempt must change the proof layer (Kore rewriting strategies, lemma structure, or a different prover consuming the same KORE), not the execution backend.

6. Tier-3 result

STATEMENT FORMALIZED; NOT YET MECHANIZED — unchanged from Phase 0, re-verified fresh under this phase's environment. Exact blocker: (i) under the Haskell backend, recursive functions over symbolic Sets do not reduce and structural induction over symbolic aggregates is unavailable (T3-U/T3-FD/T3-A residuals verbatim in `transcripts/audit/tier3_universal.txt`); (ii) under the LLVM backend, no symbolic engine exists (§5). Tier-1 (ground reconciliation, 9 claims) and tier-2 (state-generalized under `wfCoverage`) remain PROVED BY K; the falsification probes that motivated the exact `wfCoverage` premise still fail correctly (matrix [7c]).

7. KEVM LLVM binding

Compilation. `kompile k/kevm/srw3-kevm.k --backend llvm --main-module SRW3-EVM` succeeds: exit 0, 85.1 s, 2283.3 MiB peak, producing a working interpreter. The Phase-0 note that the KEVM LLVM link "requires building `libkrypto.a` (cryptopp/libff submodules)" did **not** materialize for this binding: with the plugin's K sources at the pinned SHA and no C library, the LLVM link completes. The scope caveat is unchanged from Phase 0 — no keccak-dependent opcode is exercised by the demos, and the binding's applicability is limited to that scope. The prerequisite discovery of this phase is that the binding's requires chain reaches `plugin/krypto.md` through `serialization.md`, so the pinned plugin K sources must be present in the `kproj` include path for any backend.

Architecture preserved. The binding is compiled from the unmodified `k/kevm/srw3-kevm.k`: inherited KEVM rules untouched (no EVM opcode-semantics change), SRW3 driver/syntax additions separate, and the commitment/restore mechanism threaded through `#w3State(...)` exactly as established. No architectural concession to LLVM was needed; the threaded-state design (adopted in Phase 0 to avoid cell-map regeneration problems) compiles and runs unchanged under both backends.

Execution — 3/3 demos structurally identical across backends. Each demo was run under LLVM and freshly re-run under Haskell with identical invocation; comparison is on the normalized <k> decision cell (byte-equal after whitespace normalization) and the <exit-code>:

demo	expected	Haskell	LLVM	verdict
evm_positive	COMMIT; storage {oracle:{0:100,1:1}, lending:{0:100,1:5}, liq:{0:4,1:100}}; lineage price 100; n=1; head 0	matches	matches (byte-equal cell)	IDENTICAL
evm_negative_stale	REJECT; RESTORE all storage; ln: .Map, n=0, head -1	matches	matches (byte-equal cell)	IDENTICAL
evm_min2	REJECT; RESTORE; ln: .Map, n=0, head -1	matches	matches (byte-equal cell)	IDENTICAL

The commit snapshot is produced by REAL EVM execution (real SSTORE through `evm.md` rules) under both backends, and the reject path restores the committed storages through the same `#w3State` mechanism. Evidence: `transcripts/audit/evm_llvm_equivalence.txt` (plus per-demo timing files). The pipeline real EVM execution → prospective state → SRW3 obligation evaluation → commit/reject + restore is thus demonstrated under **both** backends, with LLVM roughly 1.3–1.4× faster wall-clock on these demos (§10, Part X note).

8. Regression matrix

All Haskell-backend results were re-run fresh in this phase (matrix v4, `transcripts/audit/full_matrix_v4.txt`; the Phase-0 canonical `full_matrix_v3.txt` is preserved untouched). Where LLVM cannot execute or prove a category, the reason is stated explicitly.

#	category	expected	v4 result	LLVM
1	Python baseline suite	15/15	15/15 , exit 0	n/a (reference implementation)
2	Abstract demos	13/13	13/13 , all cells as-established	13/13, semantically equivalent (cell compare, §4)
3	Claims 1, 2, 3, 4a, 4b	PASS, #Top, no vacuity	exit 0, #Top	not attemptable — kprove is haskell-only (§5)
4	Negative control	FAILS correctly	exit 113 (correct)	same reason
5	Bridge tier 1	PASS (9 ground claims)	exit 0, #Top, 9 WarnTrivial	same reason
6	Phantom-reject defect audit	corrected=1, defective=2	reproduced	same reason
7	Bridge tier 2 (SF0-SF3; G8-G11+X3; necessity probe)	PASS; probe fails	exit 0 + #Top; probe non-zero (coverage premise load-bearing)	same reason
8	Ghost theorem (GH-BASE/A/R/SO UND-I/F/GATE-AGREE/ISO/T1/T2/M)	PASS	all PROVED, #Top (GH-T2 = $\forall n$ theorem)	same reason
9	Ghost negative control (GH-T2-NEG)	FAILS correctly	FAILED (exit 1) — non-vacuity control intact	same reason
10	KEVM positive	COMMIT	COMMIT (fresh Haskell run)	COMMIT — structurally identical

#	category	expected	v4 result	LLVM
1 1	KEVM stale-price	REJECT + RESTORE	REJECT + RESTORE	identical
1 2	KEVM minimal (min2)	REJECT + RESTORE	REJECT + RESTORE	identical
1 3	Ghost exact-fidelity certificate (new, §2.1)	PASS	PASS (13 live-code additions, 0 undeclared; selftest 4/4)	n/a (source check)
1 4	Tier-3 universal (T3-U/FD/A)	STUCK (documented)	STUCK — statement unweakened	not attemptable (§5.3)

No existing result changed: every Haskell-backend verdict reproduces the Phase-0 canonical record exactly, and every LLVM addition is either an equivalence or a documented non-applicability.

9. Updated evidence matrix

The five allowed classifications are unchanged for every Phase-0 item. No result was upgraded because LLVM executes it: execution under a second backend is a portability result, not a proof result.

item	classification	change in 1A
Claims 1–3	PROVED BY K	unchanged
Claim 4a	PROVED BY K	unchanged
Claim 4b	PROVED BY K	unchanged
Full $\forall n$ safety theorem (ghost-instrumented)	PROVED BY K	unchanged (Haskell-backend proof; LLVM has no prover)
Tier-1 reconciliation	PROVED BY K	unchanged
Tier-2 generalized reconciliation	PROVED BY K	unchanged
Tier-3 universal reconciliation	STATEMENT FORMALIZED; NOT YET MECHANIZED	blocker re-confirmed on both backends (§6)
Effect completeness	PROVED BY K (CM4)	unchanged
Interaction closure	PROVED BY K	unchanged
Structural lineage	PROVED BY K	unchanged
Cryptographic lineage	NOT YET MECHANIZED	unchanged
KEVM execution binding	DEMONSTRATED BY KEVM	strengthened: demonstrated on both backends, structurally identical (§7)
Commitment-point mechanism	DEMONSTRATED BY KEVM	unchanged
CM2	NOT YET MECHANIZED (target of Phase 0-D scope note)	unchanged
CM3	NOT YET MECHANIZED	unchanged
CM8	NOT YET MECHANIZED (hyperproperty-level decision required first)	unchanged
CM9	NOT YET MECHANIZED (hyperproperty-level decision required first)	unchanged
Network/protocol commitment enforcement	REQUIRES CLIENT/PROTOCOL SUPPORT	unchanged
LLVM backend portability (new item)	DEMONSTRATED BY KEVM (execution equivalence, both layers)	added; carries no proof weight

10. Performance measurement (Part X)

Measured on the constrained reference environment: 2 × Intel Xeon cores, 4.14 GiB RAM, Debian trixie (13.4), unprivileged execution, Java heap pinned at 2600 m (Phase-0 setting). Wall time and peak child RSS via `scripts/timeit.py (getusage(RUSAGE_CHILDREN))`. Semantics were not modified or optimized; the benchmark set is deliberately modest and reproducible.

measurement	command	wall	peak RSS	result
kompile srw3.k (Haskell)	kompile k/srw3.k --backend haskell --main-module SRW3-GATE ...	10.03 s	480.3 MiB	exit 0
kompile srw3.k (LLVM)	kompile k/srw3.k --backend llvm --main-module SRW3-GATE ...	12.60 s	377.1 MiB	exit 0
kompile srw3.k (LLVM, --enable-search)	same + --enable-search	15.38 s	458.8 MiB	exit 0
kompile binding (LLVM)	kompile k/kevm/srw3-kevm.k --backend llvm --main-module SRW3-EVM ...	85.08 s	2283.3 MiB	exit 0
abstract demo suite (Haskell, 13 demos)	bash k/run_demos.sh	33.31 s	184.2 MiB	13/13
abstract demo suite (LLVM, 13 demos)	bash scripts/llvm_demo_suite.sh	28.18 s	188.1 MiB	13/13
evm_positive (Haskell / LLVM)	krun ... -d kevm-{hs,llvm}-out	26.78 / 18.64 s	296.6 / 279.2 MiB	identical outcome
evm_negative_stale (Haskell / LLVM)	same	26.89 / 18.85 s	280.9 / 344.5 MiB	identical outcome
evm_min2 (Haskell / LLVM)	same	23.21 / 19.04 s	285.2 / 300.6 MiB	identical outcome

Interpretation: at this scale the LLVM interpreter is $\sim 1.3\text{--}1.4\times$ faster wall-clock on the EVM demos and roughly at parity on the 13 tiny abstract programs (process startup dominates there); LLVM kompile of the abstract definition is comparable to Haskell; the binding's LLVM kompile fits comfortably in the 4 GiB environment (2.28 GiB peak). Proof times are not measured per-backend because only one backend can prove (§5).

11. Scientific conclusion

- Did LLVM preserve the SRW3 semantics?** Yes. The abstract semantics compiled without any source change and all 13 demos are cell-for-cell semantically identical to the Haskell reference; the KEVM binding compiled from its unmodified source and all three demos are structurally identical (byte-equal normalized decision cell, equal exit code). No hidden rewrite was needed at either layer, and no warning class beyond the established benign ones appeared.
- Did LLVM remove any prior proof boundary?** No. All three boundaries reproduce exactly under the Haskell backend (§5), and the LLVM backend cannot attempt them: `kprove` is Haskell-only in K v7.1.337 and the LLVM backend offers concrete execution and concrete branch enumeration only. A successful `krun` under LLVM is not a proof, and none is claimed.
- Did LLVM mechanize tier-3?** No. Tier-3 remains STATEMENT FORMALIZED; NOT YET MECHANIZED with the exact blockers of §6. Its statement remains unweakened.
- Did LLVM change the status of the $\forall n$ theorem?** No. The theorem stands exactly as mechanized in Phase 0 (GH-T2/GH-M, #Top, over the ghost-instrumented semantics on the Haskell backend), and this phase re-verified the entire ghost obligation set plus the non-vacuity control with zero regression. LLVM adds no proof capability for it.
- Did LLVM reveal any new semantic discrepancy?** None. Every comparison matched (13/13 abstract, 3/3 EVM, zero regression across matrix v4). The only positive surprise was

toolchain-level: the KEVM LLVM link succeeds without building `libkrypto.a` on the K-source-only plugin path (with the unchanged keccak scope caveat), contradicting the Phase-0 expectation recorded in the prior report.

6. **What should Phase 1B investigate next?** Per the Phase-0 closure ordering, the candidates now unblocked are: CM2 (aliasing) and CM3 (aggregate limits) mechanization at the abstract layer; the cryptographic-lineage track (`libkrypto` build $\rightarrow$ keccak commitments, state-root binding, MPT inclusion proofs, then a K-level `Verify(parent, data, proof)` soundness obligation); generalized EVM programs beyond the three-contract chain; and the protocol/client layer for commitment enforcement. One meta-result should steer the proof-side planning: since a backend swap cannot move the symbolic boundaries (§5.4), tier-3 and any $\forall n$ packaging improvements must be pursued at the proof layer — e.g., proof-engine upgrades consuming the same KORE, lemma restructuring, or bounded-instantiation expansions that keep the universal statement intact.

Completion criterion. The mandate's box is satisfied on the established regression surface:

```
Haskell semantics  $\equiv$  LLVM semantics
abstract layer : 13/13 demos, decision cells equal
KEVM binding : 3/3 demos, decision cell byte-equal + exit equal
plus documented results for all three known prover boundaries (§5)
```

The strongest desired outcome (tier-3 becoming PROVED BY K) did **not** occur, and Phase 1A is still successful per the mandate's alternative clause: the exact boundary was reproduced, minimalized, and classified, with the LLVM non-applicability documented as a structural fact rather than an assumption.